

1 NN-Lite: Automated Benchmarking of PyTorch 2 Models on Android Devices

3 **Dmitry Ignatov** ¹, **Faraz Kayani** ¹, **Sarmad Kayani** ¹, and **Radu**
4 **Timofte** ¹

5 ¹ Computer Vision Lab, CAIDAS & IFI, University of Würzburg, Germany   Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 05 October 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

6 Summary

7 A modern smartphone contains several processors that can run neural networks: the central
8 processing unit (CPU), the graphics processing unit (GPU) and often a neural processing unit
9 (NPU). Researchers train their models with PyTorch ([Paszke et al., 2019](#)), and transferring
10 each model to a phone by hand becomes impractical when many models must be measured.

11 NN-Lite automates the deployment and measurement of user-supplied PyTorch vision models
12 that satisfy the documented input and conversion requirements. With an Android phone
13 connected by USB, the following procedure is used for each model: conversion to LiteRT
14 at full (FP32) and at reduced (INT8) precision is attempted and can fail. The converted
15 file is then copied to the phone. On the phone, latency is measured for the supported
16 configurations of CPU, GPU-delegate and Android Neural Networks API (NNAPI) execution.
17 These configurations are requested explicitly and recorded with their outcomes. Models come
18 from the user's own files or from the LEMUR model collection ([Goodarzi et al., 2025](#)) installed
19 with NN-Lite. A multi-day unattended run resumes after a disconnection or a restart.

20 Statement of need

21 Research on efficient deep learning (hardware-aware architecture search, quantization, latency
22 prediction, model compression) requires measurements across many architectures and real
23 devices, because an emulator does not reproduce the vendor-specific GPU and NPU software
24 stack of a phone. Some operators are not supported ([Almeida et al., 2021](#); [Ignatov et al.,](#)
25 [2019](#)). On a real device, the same model can succeed with one accelerator and be rejected by
26 another.

27 NN-Lite, which evolved from the pipeline that produced the latency dataset of Din, Hussain,
28 et al. ([2026](#)), serves researchers who need reproducible latency and feasibility measurements,
29 specific to a device and to an accelerator configuration. This paper documents release 1.0.1,
30 its design and tests, and its changes with respect to the campaign pipeline. Five of the changes
31 listed in the changelog matter most.

32 **User-supplied vision models.** The campaign pipeline operated only on the collection it was
33 written for. A file is now also accepted: a program exported with `torch.export`, which retains
34 its input shape, or a Python module with a checkpoint or the saved model. Each model
35 must have a single four-dimensional (NCHW) image input. Correct preprocessing for INT8
36 calibration remains the user's responsibility.

37 **Packaged installation.** The dependence on nightly builds that were later withdrawn from the
38 Python Package Index made the campaign impossible to reproduce. Release 1.0.1 fixes the
39 exact versions of its dependencies, all of which are published on the Python Package Index,

40 is installed under the name `nn-lit`, and its console command is `nn-lite-bench`. Linux and
41 Apple Silicon are supported.

42 **Measurements attributable to the intended model.** Models are now built at their training
43 resolution, not at a fixed size. The VisionTransformer configuration of the collection, for
44 example, loads its weights only at its training resolution of 299×299 . Verifying each transfer
45 to the phone keeps a stale file from being timed. CPU timing uses the built-in LiteRT kernels
46 with XNNPACK disabled (`--use_xnnpack=false`), as in the campaign.

47 **Quantization calibrated on representative data.** Integer export in the campaign pipeline used a
48 set of synthetic random noise. Such a set misrepresents the activation statistics and can reduce
49 the accuracy. The calibration now uses real images, processed with the input transformation
50 of the model itself.

51 **A tested programming interface.** Output parsing, error extraction and record construction are
52 separated from the device control into documented importable modules. These modules need
53 neither the deep learning frameworks nor the Android Debug Bridge (ADB). An explicit serial
54 number in every device command lets several phones be benchmarked at the same time, one
55 process per phone.

56 State of the field

57 *AI Benchmark* (Ignatov et al., 2018, 2019) and *MLPerf Mobile* (Janapa Reddi et al., 2022)
58 execute fixed curated sets of reference models in order to compare devices. Measured latency
59 serves as ground truth for latency predictors such as *nn-Meter* (L. L. Zhang et al., 2021) and
60 *BRP-NAS* (Dudziak et al., 2020), for search methods such as *MnasNet* (Tan et al., 2019) and
61 *ProxylessNAS* (Cai et al., 2019), and for benchmarks such as *HW-NAS-Bench* (Li et al., 2021).
62 *nn-Meter* includes backend preparation for Android profiling, while the other works do not
63 provide the measurement infrastructure. The closest work in spirit is Q. Zhang et al. (2022),
64 where 15 models are benchmarked with six mobile deep learning libraries on ten devices. That
65 study, however, covers a fixed set of models and is not a tool that researchers can apply to their
66 own models and devices. Compiler stacks like *TVM* (Chen et al., 2018) tune individual models.
67 *Qualcomm AI Hub* (Qualcomm Technologies, Inc., 2026) offers hosted profiling, but only on
68 devices with Qualcomm chips. Compared to *ExecuTorch* (PyTorch Foundation, 2025), which
69 provides PyTorch edge deployment workflows, NN-Lite orchestrates LiteRT campaigns over
70 many models and devices. The `benchmark_model` utility of LiteRT (Google AI Edge, 2026b)
71 measures one converted model per invocation and supports structured profiling output, but it
72 does not provide conversion, failure bookkeeping or one record for many models and devices.

73 NN-Lite uses existing components and does not replace them: LiteRT Torch (Google AI Edge,
74 2026a) for conversion, the LiteRT runtime (Abadi et al., 2016; Google AI Edge, 2025) with
75 the official `benchmark_model` binary for execution, and ADB (Android Open Source Project,
76 2026a) for device control. NNAPI (Android Open Source Project, 2026b) is used to request
77 accelerator execution. The contribution of NN-Lite consists in the orchestration layer, which
78 integrates the components into a reproducible and fault-tolerant workflow from PyTorch models
79 to structured records.

80 Software design

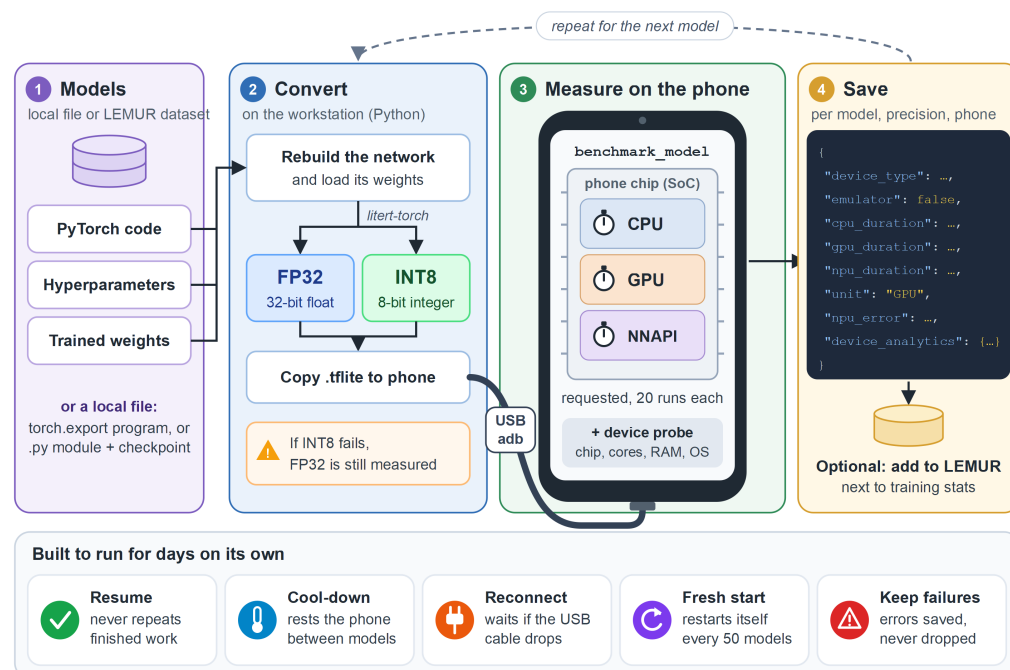


Figure 1: How NN-Lite benchmarks one model: (1) the model is read from a local file or from LEMUR (Goodarzi et al., 2025); (2) it is rebuilt and converted to FP32 and INT8 LiteRT files; (3) each file is timed on the phone with `benchmark_model` in the CPU, GPU-delegate and NNAPI configurations, which are requested, not verified on the hardware; (4) one record per model, precision and device is written. The safeguards at the bottom support unattended multi-day runs.

81 **Host-driven execution using standard Android tooling.** Transferring Google's prebuilt
82 `benchmark_model` binary once to the phone and invoking it per model and configuration avoids
83 root access and signed applications. Android tends to run processes started over ADB on the
84 slower cores of a phone, so `benchmark_model` is pinned with taskset to all cores except the
85 slowest group. End-to-end application latency is not reported.

86 **Models are rebuilt rather than assumed to be portable.** A model taken from the collection
87 consists of PyTorch code, hyperparameters and a checkpoint. NN-Lite reinstantiates the
88 network and restores its weights. A model from a file is self-describing in case of `torch.export`.
89 For every other model from a file, reconstruction from its own module avoids naming collisions
90 under a shared class name.

91 **Every configuration is measured, and failures are data.** Each model is exported in FP32. INT8
92 export with full integer post-training quantization (Jacob et al., 2018) follows for every model
93 that supports it. If quantization fails on an unsupported operator, the FP32 model is still
94 measured. Requesting every configuration explicitly, and not delegating it to the default of
95 the runtime, means that each latency is labelled with the requested backend. Recording the
96 diagnostic output of a failed configuration in the result and in the per-device log, and retaining
97 a model whose configurations all fail and marking it invalid, makes failures available for explicit
98 analysis. Not every failure establishes an operator support gap.

99 **Fault-tolerant, resumable runs.** NN-Lite pauses between models, restarts its process from time
100 to time to release the memory held by the conversion toolchain, with a longer pause for cooling
101 at each restart, and waits for a disconnected device.

102 **One schema for every model source.** Results are written as one file per model, quantization
103 setting and device, in the training statistics layout of the collection. Each record contains the
104 mean, minimum, maximum and standard deviation of the latency over repeated executions for
105 every configuration, the fastest configuration (the one with the lowest valid mean latency), and
106 the input shape, memory state and device telemetry needed for interpretation. The requested
107 number of timed runs (20 by default) is stored as iterations. Each configuration performs
108 exactly this number of runs after a warm-up, and the statistics are computed from these runs
109 only.

110 Two limitations have to be mentioned. NN-Lite reports wall-clock latency and does not isolate
111 the frequency governor state or the energy per operator. The second limitation is that NPU
112 execution is mediated by NNAPI, whose behaviour depends on the installed runtime and drivers:
113 NNAPI can fall back to CPU execution when compilation or execution on an accelerator fails,
114 and the API was deprecated in Android 15 ([Android Open Source Project, 2026b](#)). Because
115 NN-Lite cannot guarantee that all operations ran on the intended accelerator, the label NPU
116 in the schema denotes the NNAPI configuration. The command line of each configuration is
117 built in a single function, and this function localizes most of the change for a replacement
118 backend.

119 Research impact statement

120 NN-Lite is the on-device measurement component of the LEMUR ecosystem ([Goodarzi et al., 2025](#)).
121 Inference latency measurements for more than 7,500 models in LEMUR 2 ([Uzun et al., 2026](#))
122 were provided by the emulator execution path of NN-Lite. The physical device path
123 produced the delegate-resolved latency dataset reported in Din, Hussain, et al. (2026), which
124 covers 586 architectures across five commodity phones from three chip vendors. All records
125 of this dataset are publicly available. They have also been used to fit device-specific latency
126 models by symbolic regression ([Dhanani et al., 2026](#)) and to show that FLOPs predict latency
127 poorly if the backend is not taken into account ([Din, Kayani, et al., 2026](#)).

128 Release 1.0.1 is on the Python Package Index as `nn-lit`, with the source in the project
129 repository. Unit tests cover the release and run in continuous integration on three Python
130 versions. We verified both model sources end to end on a phone, including a model outside
131 the collection (torchvision MobileNetV3).

132 AI usage disclosure

133 We designed and implemented the NN-Lite software without generative AI assistance, while
134 Claude Opus 5.5 (Anthropic) assisted with drafting the manuscript, checking paper.bib and
135 creating [Figure 1](#). All the content produced with AI assistance was reviewed, edited and
136 validated by the authors, who made all the final scientific and software decisions.

137 Acknowledgements

138 This work was partially supported by the Alexander von Humboldt Foundation. We thank Saif
139 U Din and Muhammad Ahsan Hussain for the measurement campaign of Din, Hussain, et al.
140 ([2026](#)), and the contributors to the LEMUR dataset.

141 References

142 Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S.,
143 Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G.,
144 Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). TensorFlow: A

- 145 system for large-scale machine learning. *12th USENIX Symposium on Operating Systems*
 146 *Design and Implementation (OSDI)*, 265–283. <https://doi.org/10.48550/arXiv.1605.08695>
- 147 Almeida, M., Laskaridis, S., Mehrotra, A., Dudziak, L., Leontiadis, I., & Lane, N. D. (2021).
 148 Smart at what cost? Characterising mobile deep neural networks in the wild. *Proceedings*
 149 *of the 21st ACM Internet Measurement Conference (IMC)*, 658–672. [https://doi.org/10.](https://doi.org/10.1145/3487552.3487863)
 150 [1145/3487552.3487863](https://doi.org/10.1145/3487552.3487863)
- 151 Android Open Source Project. (2026a). *Android Debug Bridge (adb)*. [https://developer.](https://developer.android.com/tools/adb)
 152 [android.com/tools/adb](https://developer.android.com/tools/adb).
- 153 Android Open Source Project. (2026b). *Neural Networks API*. [https://developer.android.com/](https://developer.android.com/ndk/guides/neuralnetworks)
 154 [ndk/guides/neuralnetworks](https://developer.android.com/ndk/guides/neuralnetworks).
- 155 Cai, H., Zhu, L., & Han, S. (2019). ProxylessNAS: Direct neural architecture search on
 156 target task and hardware. *International Conference on Learning Representations (ICLR)*.
 157 <https://doi.org/10.48550/arXiv.1812.00332>
- 158 Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu,
 159 Y., Ceze, L., Guestrin, C., & Krishnamurthy, A. (2018). TVM: An automated end-to-end
 160 optimizing compiler for deep learning. *13th USENIX Symposium on Operating Systems*
 161 *Design and Implementation (OSDI)*, 578–594. <https://doi.org/10.48550/arXiv.1802.04799>
- 162 Dhanani, D. R., Kayani, F., Din, S. U., Arslanian, A., Ignatov, D., & Timofte, R. (2026).
 163 Understanding the performance of deep computer vision models: A symbolic regression
 164 approach to accuracy and latency prediction. *Sensors*, 26(13), 4093. [https://doi.org/10.](https://doi.org/10.3390/s26134093)
 165 [3390/s26134093](https://doi.org/10.3390/s26134093)
- 166 Din, S. U., Hussain, M. A., Ikram, M., Kayani, F., Ignatov, D., & Timofte, R. (2026). AI on
 167 the edge: An automated pipeline for PyTorch-to-Android deployment and benchmarking.
 168 *Preprints*. <https://doi.org/10.20944/preprints202511.1831.v2>
- 169 Din, S. U., Kayani, F., Timofte, R., & Ignatov, D. (2026). Vision model inference on mobile
 170 devices: A large-scale delegate-aware benchmark beyond FLOPs. *Proceedings of the British*
 171 *Machine Vision Conference (BMVC)*.
- 172 Dudziak, L., Chau, T., Abdelfattah, M. S., Lee, R., Kim, H., & Lane, N. D. (2020). BRP-NAS:
 173 Prediction-based NAS using GCNs. *Advances in Neural Information Processing Systems*
 174 *(NeurIPS)*, 33. <https://doi.org/10.48550/arXiv.2007.08668>
- 175 Goodarzi, A. T., Kochnev, R., Khalid, W., Goudarzi, H. T., Qin, F., Uzun, T. A., Dhameliya,
 176 Y. S., Kathiriya, Y. K., Benty, Z. A., Ignatov, D., & Timofte, R. (2025). LEMUR
 177 neural network dataset: Towards seamless AutoML. *arXiv Preprint arXiv:2504.10552*.
 178 <https://doi.org/10.48550/arXiv.2504.10552>
- 179 Google AI Edge. (2025). *LiteRT: On-device runtime and developer guide*. [https://ai.google.](https://ai.google.dev/edge/litert)
 180 [dev/edge/litert](https://ai.google.dev/edge/litert).
- 181 Google AI Edge. (2026a). *LiteRT Torch (formerly AI Edge Torch): Converting PyTorch*
 182 *models to LiteRT*. <https://github.com/google-ai-edge/litert-torch>.
- 183 Google AI Edge. (2026b). *LiteRT: Benchmark interpreter API*. [https://developers.google.](https://developers.google.com/edge/litert/models/measurement)
 184 [com/edge/litert/models/measurement](https://developers.google.com/edge/litert/models/measurement).
- 185 Ignatov, A., Timofte, R., Chou, W., Wang, K., Wu, M., Hartley, T., & Van Gool, L. (2018).
 186 AI Benchmark: Running deep neural networks on Android smartphones. *Proceedings*
 187 *of the European Conference on Computer Vision (ECCV) Workshops*, 288–314. https://doi.org/10.1007/978-3-030-11021-5_19
- 188
 189 Ignatov, A., Timofte, R., Kulik, A., Yang, S., Wang, K., Baum, F., Wu, M., Xu, L., &
 190 Van Gool, L. (2019). AI Benchmark: All about deep learning on smartphones in 2019.
 191 *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*

- (ICCVW), 3617–3635. <https://doi.org/10.1109/ICCVW.2019.00447>
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713. <https://doi.org/10.1109/CVPR.2018.00286>
- Janapa Reddi, V., Kanter, D., Mattson, P., Duke, J., Nguyen, T., Chukka, R., Shiring, K., Tan, K.-S., Charlebois, M., Chou, W., El-Khamy, M., Hong, J., St John, T., Trinh, C., Buch, M., Mazumder, M., Markovic, R., Atta, T., Cakir, F., ... Zika, D. (2022). MLPerf Mobile inference benchmark: An industry-standard open-source machine learning benchmark for on-device AI. *Proceedings of Machine Learning and Systems (MLSys)*, 4, 352–369. https://proceedings.mlsys.org/paper_files/paper/2022/hash/a2b2702ea7e682c5ea2c20e8f71efb0c-Abstract.html
- Li, C., Yu, Z., Fu, Y., Zhang, Y., Zhao, Y., You, H., Yu, Q., Wang, Y., & Lin, Y. (2021). HW-NAS-Bench: Hardware-aware neural architecture search benchmark. *International Conference on Learning Representations (ICLR)*. <https://doi.org/10.48550/arXiv.2103.10584>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32. <https://doi.org/10.48550/arXiv.1912.01703>
- PyTorch Foundation. (2025). *ExecuTorch: On-device inference runtime for PyTorch models*. <https://docs.pytorch.org/executorch/>.
- Qualcomm Technologies, Inc. (2026). *Qualcomm AI Hub*. <https://aihub.qualcomm.com>.
- Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., & Le, Q. V. (2019). MnasNet: Platform-aware neural architecture search for mobile. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2815–2823. <https://doi.org/10.1109/CVPR.2019.00293>
- Uzun, T. A., Khalid, W., Din, S. U., Mulukuledu, S. R., Singh, A., Vysyaraju, C., Duvvuri, R., Goyal, A., Lukhi, Y. R., Hussain, M. A., Jesani, K., Shrestha, U., Mittal, Y., Kochnev, R., Kadam, P., Ikram, M., Moradiya, H. R., Arslanian, A., Ignatov, D., & Timofte, R. (2026). LEMUR 2: Unlocking neural network diversity for AI. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3291–3300. https://openaccess.thecvf.com/content/CVPR2026W/CVPR-NAS26/papers/Uzun_LEMUR_2_Unlocking_Neural_Network_Diversity_for_AI_CVPRW_2026_paper.pdf
- Zhang, L. L., Han, S., Wei, J., Zheng, N., Cao, T., Yang, Y., & Liu, Y. (2021). nn-Meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices. *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 81–93. <https://doi.org/10.1145/3458864.3467882>
- Zhang, Q., Li, X., Che, X., Ma, X., Zhou, A., Xu, M., Wang, S., Ma, Y., & Liu, X. (2022). A comprehensive benchmark of deep learning libraries on mobile devices. *Proceedings of the ACM Web Conference (WWW)*, 3298–3307. <https://doi.org/10.1145/3485447.3512148>